

Detecting and Analyzing Timestomping

Using KAPE and Timeline Explorer

Digital Forensics & Incident Response

By Julian Derry

June 2026

1. Introduction

Timestamping is an anti-forensic technique used by threat actors to manipulate NTFS timestamps on a Windows file system. By forging a file's timestamps, an attacker can make a newly created malicious tool appear as though it has existed on a system for years. When an analyst filters artifacts by the incident timeframe, the stomped file falls outside that window entirely but hidden in plain sight.

This document outlines how to detect and analyze timestamping using:

- KAPE (Kroll Artifact Parser and Extractor) for artifact acquisition
- MFTECmd for parsing the \$MFT into analyzable CSV output
- Timeline Explorer for identifying indicators of manipulation

2. Simulating Timestamping

2.1 Original File Timestamps

A test file was created with the following original timestamps:

- Date Created: 6/25/2026 – 11:51 AM
- Date Modified: 6/25/2026 – 11:51 AM

Original file timestamps before modification

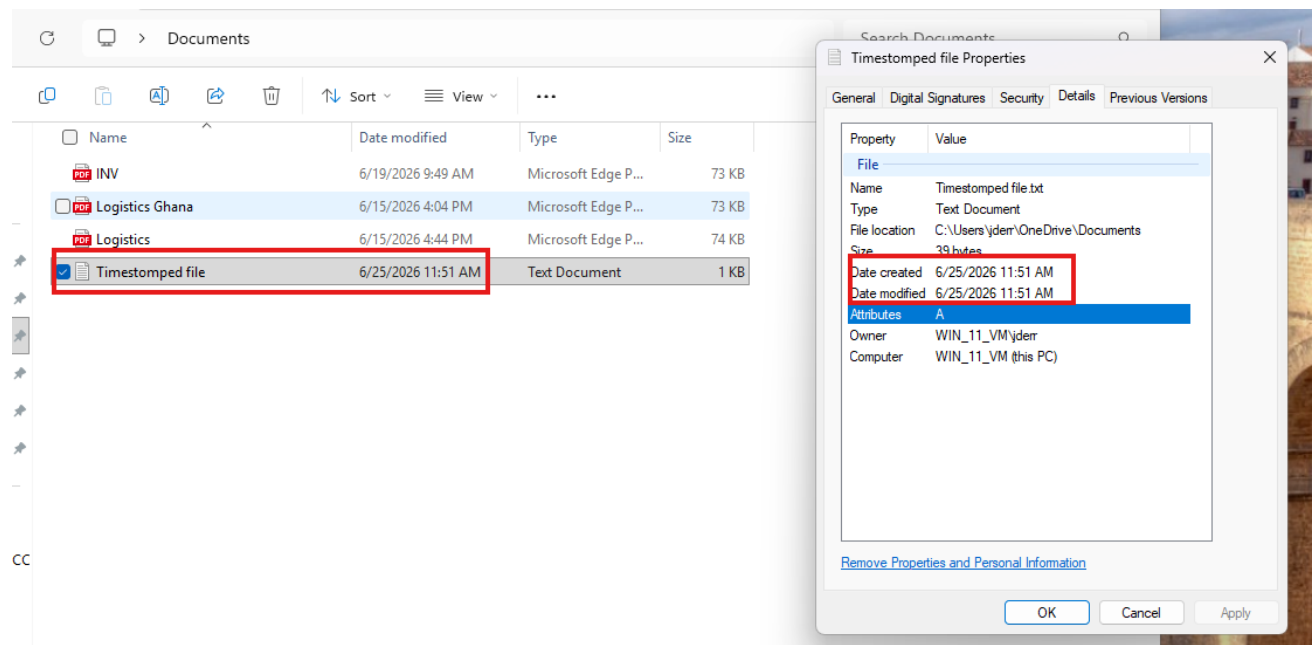


Figure 1 – Original file timestamps before modification

2.2 Stomping the Timestamps

Both timestamps were manipulated using PowerShell — a method readily available to any attacker on a Windows system.

Stomp the Last Write (Modified) time:

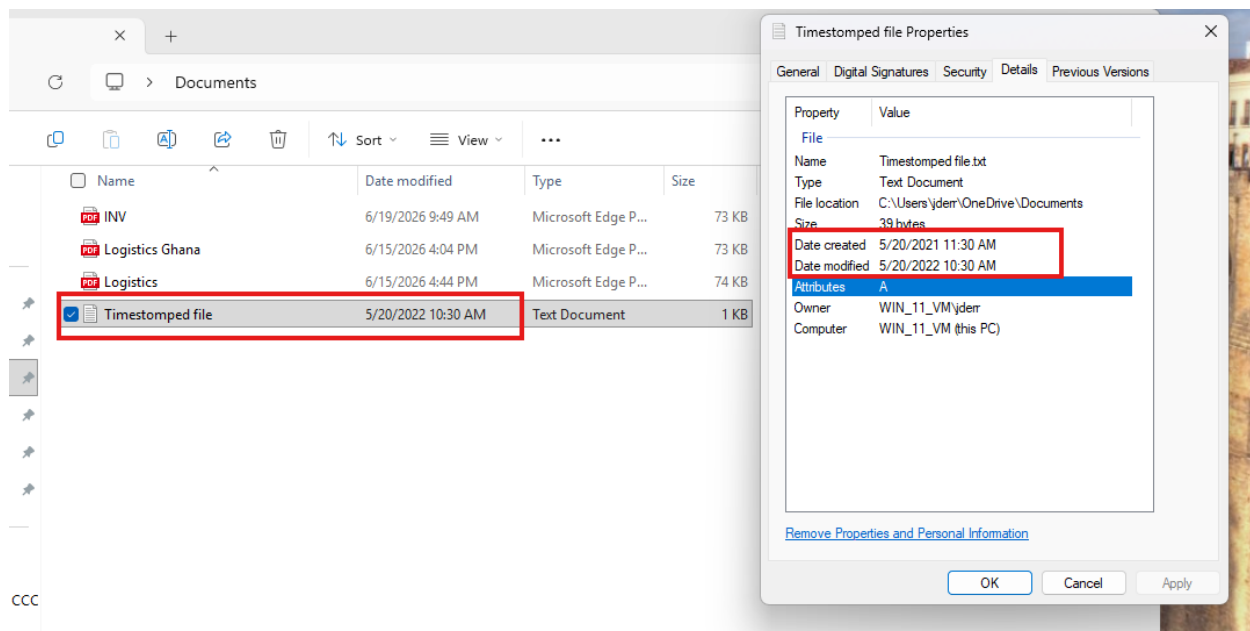
```
powershell -command "(Get-Item  
'C:\Users\jderr\OneDrive\Documents\Timestamped file.txt').LastWriteTime =  
'05/20/2022 10:30 AM'"
```

Stomp the Creation time:

```
powershell -command "(Get-Item  
'C:\Users\jderr\OneDrive\Documents\Timestamped file.txt').CreationTime =  
'05/20/2021 11:30 AM'"
```

The file's timestamps now appear to predate the incident by several years, making it indistinguishable from pre-existing system files during a standard timeline review.

Figure 2 – File timestamps after timestamping via PowerShell



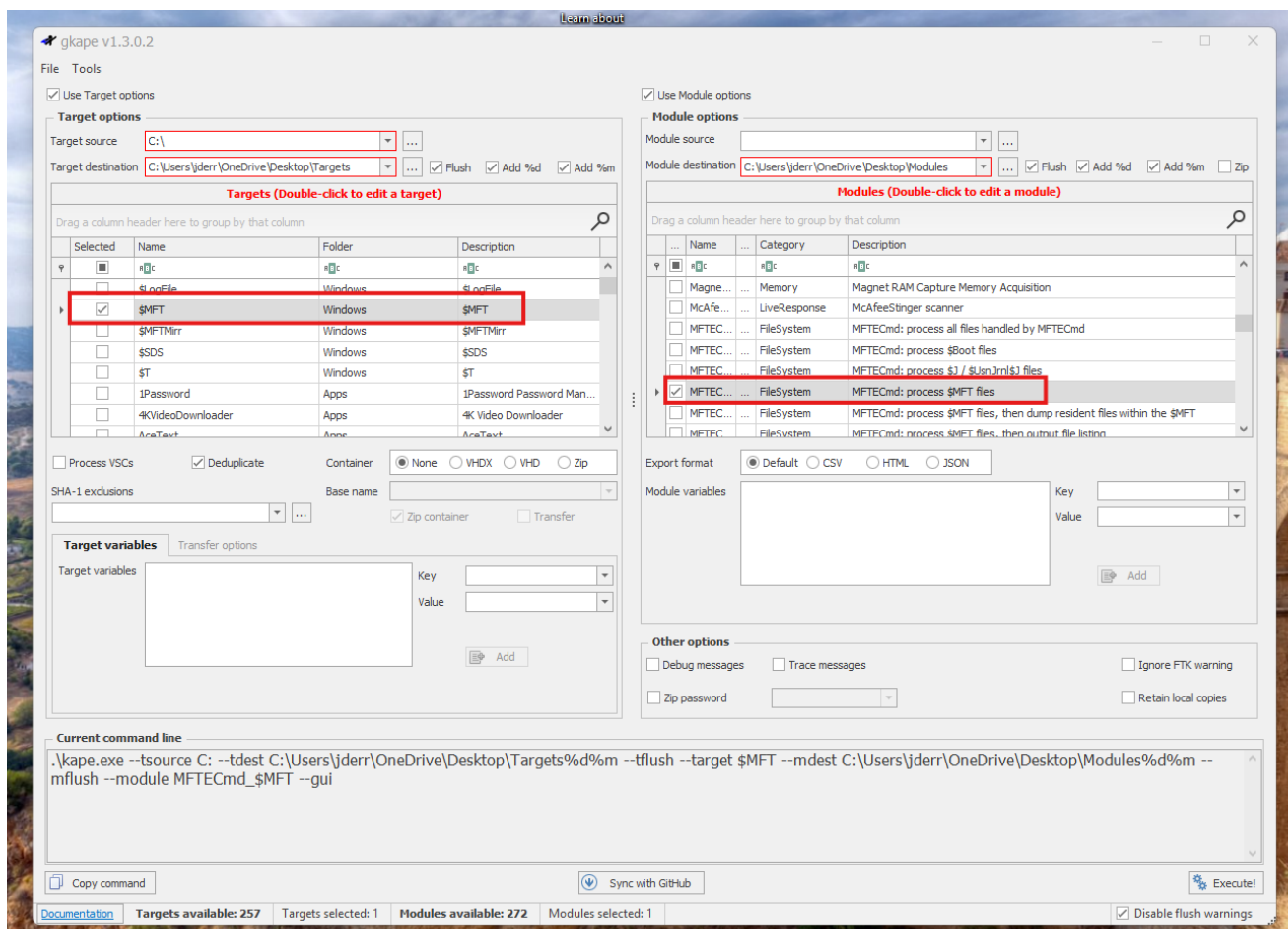
3. Collecting the \$MFT with KAPE

The \$MFT (Master File Table) is a core NTFS metadata file that maintains two independent sets of timestamps for every file and folder on a volume. Because it retains records that user-level processes cannot easily modify, it is the primary artifact for detecting timestamping.

Steps to collect and parse the \$MFT using gKAPE:

- Open gkape.exe and set the Target Source to the C:\ drive.
- Set a destination folder where the \$MFT artifact will be stored.
- Under Targets, enable the \$MFT option.
- Under Modules, select the MFTECmd_\$MFT Module to automatically parse the \$MFT into CSV output.
- Run KAPE. The parsed CSV will appear in the defined Module destination folder.

gKAPE configuration for \$MFT collection and parsing



4. Identifying Indicators in Timeline Explorer

Load the parsed \$MFT CSV into Timeline Explorer by dragging and dropping the file into the application. Search for the file by name or extension. Three indicators point to timestomping:

4.1 Timestamp Mismatch (SI < FN)

Every NTFS file carries two sets of timestamps: \$STANDARD_INFORMATION (0x10), which Windows displays to users and which user-level processes can modify, and \$FILE_NAME (0x30), which is maintained by the Windows kernel and cannot be easily altered by attackers.

When the user-facing timestamp (0x10) is earlier than the kernel-level record (0x30), that backwards gap is flagged as SI < FN in MFTECmd output and is the primary fingerprint of a file stomped into the past. Note: this logic will not catch the rare case where a file is stomped into the future, though a future-dated file should stand out on its own.

SI < FN flag in Timeline Explorer confirming timestamp discrepancy

File Name	Extension	I...	Ha...	Is...	Fi...	Created0x10	Created0x30	Last Modified0x10	Las...	L...	L...	La...	Z...	Re...	R...	SI<FN	u	Sec	Zeros	Copied
					=	=	=	=	=	=	=	=			=					
Timestomped file.lnk	.lnk				704	2026-06-25 10:51:32		2026-06-25 10:55:05	2...	2...					1					
Timestomped file.txt	.txt				39	2021-05-20 10:30:00	2026-06-25 10:51:32	2022-05-20 09:30:00	202...	2...	2...	20...			1					

4.2 Zeroed Subseconds on Created (0x10)

Real file timestamps carry nanosecond precision. Most timestomping tools cannot set this level of detail, so the subsecond field is wiped to .0000000. A Created (0x10) timestamp ending in .00000000 is a strong secondary indicator, though it should be weighed alongside other evidence, some legitimate archiving tools can also produce zeroed subseconds.

Zeroed subseconds and Created (0x30) value visible in Timeline Explorer output

File Name	Extension	Is Directory	Has Ads	Is Ads	File Size	Created0x10	Created0x30
⚡	⚡	☐	☐	☐	=	=	=
Timestomped file.lnk	.lnk	☐	☐	☐	704	2026-06-25 10:51:32	
Timestomped file.txt	.txt	☐	☐	☐	39	2021-05-20 10:30:00	2026-06-25 10:51:32

The presence of Created(0x10) timestamp is usually an indicator timestomping. Created (0x10) timestamp showing .00000000 milliseconds appended to the end of the file's timestamp.

The presence of Created(0x30) timestamp is useful in spotting discrepancies between two timestamp values. Created(0x30) timestamp having an entry in the MFTECmd CSV output.

Modified Created Date

Original Created Date

4.3 Value Present in Created (0x30)

In MFTECmd's CSV output, the Created (0x30) column is only populated when a discrepancy exists between the 0x10 and 0x30 timestamp values. A value in this column confirms that the \$FILE_NAME record differs from the \$STANDARD_INFORMATION record — a key indicator of manipulation.

Note: The SI < FN and u Sec Zeroes indicators are specific to MFTECmd's parsed CSV output and are not visible in the raw \$MFT itself.

The presence of Created(0x10) timestamp is usually an indicator timestamping. Created (0x10) timestamp showing .0000000 milliseconds appended to the end of the file's timestamp.

The presence of Created(0x30) timestamp is useful in spotting discrepancies between two timestamp values. Created(0x30) timestamp having an entry in the MFTECmd CSV output.

File Name	Extension	Is Directory	Has Ads	Is Ads	File Size	Created0x10	Created0x30
⚠️	⚠️	☐	☐	☐	=	=	=
Timestomped file.lnk	.lnk	☐	☐	☐	704	2026-06-25 10:51:32	
Timestomped file.txt	.txt	☐	☐	☐	39	2021-05-20 10:30:00	2026-06-25 10:51:32

Modified Created Date

Original Created Date

5. Analyst Notes

5.1 Attacker Behavior

Threat actors almost always stomp timestamps into the past rather than the future. The objective is to bury malicious files among benign pre-incident data so they appear to predate the compromise window and avoid scrutiny during triage. A timestomped file may not surface without specifically filtering for SI < FN mismatches or zeroed subseconds.

When both zeroed subseconds and a Created (0x30) value are present on the same file, the totality of circumstances strongly supports a conclusion of deliberate timestamp manipulation.

5.2 Timezone Handling

Never adjust the timezone on the system under investigation. Time normalization must always be performed on the analyst's own workstation or forensic tool not on the target.

- **Configure forensic tools such as Timeline Explorer and the analysis workstation to operate in UTC.** Normalize to UTC:

- **Extract the target's timezone from its registry files to understand user activity context, then perform all correlation in UTC.** Document the Offset:
-

6. Tools Referenced

- KAPE (Kroll Artifact Parser and Extractor) developed by Eric Zimmerman, Kroll
 - MFTECmd Eric Zimmerman's command-line \$MFT parser
 - Timeline Explorer Eric Zimmerman's CSV analysis tool
-

End of Document